

Notebook: logistic_regression/logistic_regression_from_scratch.ipynb

[--- MARKDOWN CELL ---]

Logistic Regression From Scratch

This notebook implements logistic regression from scratch using gradient descent.
We'll use the Breast Cancer dataset for binary classification.

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

[--- CODE CELL ---]

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

[--- CODE CELL ---]

```
np.random.seed(42)
```

[--- MARKDOWN CELL ---]

Load and Prepare Data

[--- CODE CELL ---]

```
# Load Breast Cancer dataset
data = load_breast_cancer()
X = data.data
y = data.target

print(f"Feature names: {data.feature_names}")
print(f"Target names: {data.target_names}")
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
```

[--- CODE CELL ---]

```
# Create a DataFrame for exploratory analysis
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y
df.head()
```

[--- CODE CELL ---]

```
df.describe()
```

[--- CODE CELL ---]

```
# Visualize class distribution
plt.figure(figsize=(8, 5))
sns.countplot(x='target', data=df)
plt.title("Class Distribution (0=Malignant, 1=Benign)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

[--- CODE CELL ---]

```
# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize features (important for gradient descent)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"X_train shape: {X_train_scaled.shape}")
print(f"X_test shape: {X_test_scaled.shape}")
```

[--- MARKDOWN CELL ---]

Logistic Regression Implementation From Scratch

[--- CODE CELL ---]

```
def sigmoid(z):
    """Sigmoid activation function."""
    # Clip z to avoid overflow in exp
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))
```

[--- CODE CELL ---]

```
def compute_cost(X, y, w, b):
    """Compute the binary cross-entropy loss (cost function)."""
    m = len(y)
    z = np.dot(X, w) + b
    y_pred = sigmoid(z)
    # Add small epsilon to avoid log(0)
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    cost = (-1/m) * np.sum(y * np.log(y_pred) + (1 - y) * np.log(1 - y_pred))
    return cost
```

[--- CODE CELL ---]

```
def compute_gradients(X, y, w, b):
    """Compute gradients for weights and bias."""
    m = len(y)
    z = np.dot(X, w) + b
    y_pred = sigmoid(z)
    # Gradient of loss w.r.t weights and bias
    error = y_pred - y
    dw = (1/m) * np.dot(X.T, error)
    db = (1/m) * np.sum(error)
    return dw, db
```

[--- CODE CELL ---]

```
def gradient_descent(X, y, w, b, lr=0.01, epochs=1000):
    """Perform gradient descent optimization."""
    cost_history = []
    for i in range(epochs):
        dw, db = compute_gradients(X, y, w, b)
        w = w - lr * dw
        b = b - lr * db
        cost = compute_cost(X, y, w, b)
        cost_history.append(cost)
        if i % 100 == 0:
            print(f"Epoch {i}: Cost = {cost:.6f}")
```

```
return w, b, cost_history
```

[--- CODE CELL ---]

```
def predict(X, w, b, threshold=0.5):
    """Make predictions using learned weights and bias."""
    z = np.dot(X, w) + b
    y_pred_prob = sigmoid(z)
    y_pred = (y_pred_prob >= threshold).astype(int)
    return y_pred, y_pred_prob
```

[--- MARKDOWN CELL ---]

Train the Model

[--- CODE CELL ---]

```
# Initialize weights and bias
n_features = X_train_scaled.shape[1]
w_init = np.zeros(n_features)
b_init = 0

# Train using gradient descent
w, b, cost_history = gradient_descent(
    X_train_scaled,
    y_train,
    w_init,
    b_init,
    lr=0.1,
    epochs=1000
)

print(f"\nFinal weights shape: {w.shape}")
print(f"Final bias: {b:.6f}")
```

[--- MARKDOWN CELL ---]

Evaluate the Model

[--- CODE CELL ---]

```
# Make predictions on test set
y_pred_custom, y_pred_prob = predict(X_test_scaled, w, b)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred_custom)
print(f"Accuracy (From Scratch): {accuracy:.4f}")
```

[--- CODE CELL ---]

```
# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_custom, target_names=['Malignant', 'Benign']))
```

[--- CODE CELL ---]

```
# Plot confusion matrix
cm = confusion_matrix(y_test, y_pred_custom)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Malignant', 'Benign'],
            yticklabels=['Malignant', 'Benign'])
plt.title("Confusion Matrix - Logistic Regression From Scratch")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

[--- CODE CELL ---]

```
# Plot training loss curve
plt.figure(figsize=(10, 6))
plt.plot(cost_history)
plt.xlabel("Epochs")
plt.ylabel("Cost (Binary Cross-Entropy)")
plt.title("Training Loss Curve - Logistic Regression From Scratch")
plt.grid(True, alpha=0.3)
plt.show()
```

[--- CODE CELL ---]

```
# Cost is decreasing, hence we know that gradient descent is working
```

[--- MARKDOWN CELL ---]

```
## Feature Importance Visualization
```

[--- CODE CELL ---]

```
# Plot feature importance (based on absolute weight values)
feature_importance = np.abs(w)
sorted_idx = np.argsort(feature_importance)[::-1][:10] # Top 10 features
plt.figure(figsize=(12, 6))
plt.barh(range(10), feature_importance[sorted_idx])
plt.yticks(range(10), [data.feature_names[i] for i in sorted_idx])
plt.xlabel("Absolute Weight Value")
plt.title("Top 10 Feature Importance (Logistic Regression)")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```